MANUAL DE

BASES DE DATOS Y DBMS

Arquitecturas, Gestores y Aplicabilidad Tecnológica

I. INTRODUCCIÓN A LOS GESTORES DE BASE DE DATOS (DBMS)

El tratamiento adecuado de la información es un factor esencial en el desarrollo de cualquier organización, ya sea empresarial, académica, social o gubernamental. Para que los datos tengan un verdadero valor, deben ser almacenados, organizados e integrados de manera eficiente.

Los gestores de bases de datos (DBMS, Database Management Systems) constituyen herramientas fundamentales, al permitir administrar la información de forma estructurada, segura y escalable.

1. Definición y Propósito de un DBMS

Un DBMS es un software especializado que permite definir, crear, manipular y administrar bases de datos. Su función principal es actuar como intermediario entre los datos almacenados y los usuarios o aplicaciones que los utilizan, asegurando que la información pueda consultarse y modificarse de manera consistente y segura.

Propósito central de un DBMS

- **Definición de estructuras de datos:** creación de tablas, relaciones, índices y esquemas.
- **Manipulación de datos:** consultas, inserciones, actualizaciones y eliminaciones.
- **Gestión de seguridad:** asignación de privilegios de acceso y control de usuarios.
- **Garantía de integridad:** aplicación de reglas y restricciones que aseguren la coherencia de la información.
- **Soporte de concurrencia:** manejo del acceso simultáneo de múltiples usuarios.
- **Mecanismos de recuperación:** restauración del sistema tras fallos o pérdidas de datos.

2. Evolución Histórica de los DBMS

El desarrollo de los gestores de bases de datos refleja la evolución de las necesidades de la sociedad en relación con la información:

Período	Hitos Principales
1960s	**Primeros modelos jerárquicos y en red —** IMS de IBM y IDMS. Eficientes para estructuras simples, pero rígidos.
1970s	**Modelo relacional —** Edgar F. Codd (IBM) propone organizar datos en tablas. Se introduce SQL.
1980-90s	**RDBMS comerciales —** Oracle Database, DB2 y SQL Server. Surge MySQL y PostgreSQL como open source.
Siglo XXI	**NoSQL y la nube —** MongoDB, Cassandra, Neo4j para datos no estructurados. Amazon RDS, Google Cloud SQL, MongoDB Atlas.

3. Tipología de Gestores: Relacionales, No Relacionales e Híbridos

Relacionales (RDBMS)	No Relacionales (NoSQL)	Híbridos / Multimodelo
Basados en modelo relacional de Codd. Datos en tablas con relaciones. **Ejemplos:** Oracle, SQL Server, MySQL, PostgreSQL **Ventajas:** Integridad, transacciones ACID, SQL estándar	Para datos no estructurados o semiestructurados. Tipos: documentos, clave-valor, grafos, columnas. **Ejemplos:** MongoDB, Redis, Neo4j, Cassandra **Ventajas:** Escalabilidad horizontal, flexibilidad de esquemas	Integran lo mejor de los modelos relacionales y no relacionales. **Ejemplos:** ArangoDB, OrientDB, Oracle Multimodel **Ventajas:** Adaptabilidad y reducción de fragmentación

4. Importancia de los DBMS en la Administración de la Información

Los DBMS cumplen una función estratégica en la gestión de la información:

Función	Descripción
Garantizan eficiencia	Permiten almacenar y recuperar información en segundos.
Mejoran la seguridad	Ofrecen mecanismos de autenticación y autorización.
Aseguran integridad	Aplican restricciones que reducen inconsistencias.
Favorecen la colaboración	Múltiples usuarios pueden trabajar simultáneamente.
Soportan decisiones	Proporcionan datos confiables para reportes y análisis.
Se adaptan al crecimiento	Soportan grandes volúmenes en arquitecturas distribuidas.

II. CARACTERÍSTICAS DE LOS PRINCIPALES GESTORES DE BASES DE DATOS RELACIONALES

Los gestores de bases de datos relacionales (RDBMS) constituyen una de las herramientas más importantes para el almacenamiento, organización, acceso y administración de la información en entornos corporativos, académicos e institucionales. Entre los más destacados se encuentran SQL Server, MySQL, PostgreSQL y Oracle Database.

1. SQL Server (Microsoft)

Microsoft SQL Server es un sistema de gestión de bases de datos relacional desarrollado por Microsoft. Es uno de los productos líderes en entornos empresariales debido a su integración con la suite de herramientas de Microsoft, su alta seguridad y escalabilidad.

Características Principales	Aplicaciones Típicas
• Lenguaje estándar SQL con extensiones T-SQL • Integración nativa con Excel, Power BI, .NET • Herramientas de administración (SSMS) • Alta disponibilidad (AlwaysOn, clustering) • Seguridad robusta con autenticación de Windows	Sistemas de facturación, CRM empresarial, análisis de datos en entornos corporativos Windows.

Ejemplo de consulta T-SQL:

```
-- Seleccionar clientes con más de 5 pedidos
SELECT c.nombre, COUNT(p.id) AS total_pedidos
FROM Clientes c
INNER JOIN Pedidos p ON c.id = p.cliente_id
GROUP BY c.nombre
HAVING COUNT(p.id) > 5
ORDER BY total_pedidos DESC;
```

2. MySQL

MySQL, desarrollado inicialmente por MySQL AB y actualmente gestionado por Oracle Corporation, es uno de los RDBMS más populares en el mundo gracias a su código abierto y su facilidad de uso. Es ampliamente utilizado en aplicaciones web y proyectos de software libre.

Características Principales	Aplicaciones Típicas
• Lenguaje estándar SQL • Gratuito (Community) y con licencias comerciales (Enterprise) • Compatible con Windows, Linux, macOS • Alta velocidad de consultas • Integración con PHP, Python, Node.js	CMS como WordPress, Joomla, Drupal. Stack LAMP (Linux, Apache, MySQL, PHP). Aplicaciones web de todo tipo.

Ejemplo de consulta MySQL:

```
-- Crear tabla de productos con índice
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(10,2),
  stock INT DEFAULT 0,
  INDEX idx_nombre (nombre)
);

-- Insertar y consultar
INSERT INTO productos (nombre, precio, stock)
VALUES ('Laptop', 1200.00, 50);

SELECT * FROM productos WHERE precio < 1500 ORDER BY precio ASC;
```

3. PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional y orientado a objetos, de código abierto, altamente valorado por su robustez, extensibilidad y cumplimiento de estándares SQL. Se considera uno de los RDBMS más avanzados.

Características Principales	Aplicaciones Típicas
• Cumplimiento estricto de estándares SQL • Soporte para tipos avanzados: JSON, XML, arrays • Extensión de funciones, operadores y tipos definidos por el usuario • Seguridad avanzada: GSSAPI, SSPI, SSL • Escalabilidad vertical y horizontal, ideal para Big Data	Sistemas financieros, aplicaciones de análisis de datos, servicios en la nube, proyectos científicos y aplicaciones GIS.

Ejemplo de consulta PostgreSQL (con JSON):

```
-- PostgreSQL soporta JSON nativo
CREATE TABLE estudiantes (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100),
  datos_extra JSONB
);

INSERT INTO estudiantes (nombre, datos_extra)
VALUES ('Ana García', '{"carrera": "Sistemas", "notas": [18,19,20]}');

-- Consultar campo JSON
SELECT nombre, datos_extra->>'carrera' AS carrera
FROM estudiantes
WHERE datos_extra->>'carrera' = 'Sistemas';
```

4. Oracle Database

Oracle Database es un RDBMS desarrollado por Oracle Corporation, reconocido como uno de los más potentes y utilizados en grandes organizaciones a nivel mundial. Está diseñado para entornos que requieren altos niveles de disponibilidad, seguridad y escalabilidad.

Características Principales	Aplicaciones Típicas
• Lenguaje PL/SQL (extensión de SQL para procedimientos) • Optimización avanzada de consultas y almacenamiento • Alta disponibilidad con Oracle RAC • Seguridad integral: cifrado, auditorías, control granular • Soporte multimodelo (relacional, documental, espacial)	Bancos, gobiernos y grandes corporaciones que manejan información crítica a nivel mundial.

Cuadro Comparativo: SQL Server vs MySQL vs PostgreSQL vs Oracle

Criterio	SQL Server	MySQL	PostgreSQL	Oracle
Licencia	Comercial / Express	Open Source / Enterprise	Open Source	Comercial
Costo	Alto (Enterprise)	Bajo / Gratuito	Gratuito	Muy Alto
Lenguaje	T-SQL	SQL estándar	SQL + extensiones	PL/SQL
Escalabilidad	Alta	Media-Alta	Alta	Muy Alta
Seguridad	Muy Alta	Media	Alta	Muy Alta
Plataforma	Windows/Linux	Multi-plataforma	Multi-plataforma	Multi-plataforma
Mejor para	Corporativos Windows	Web / Open Source	Big Data / Ciencia	Grandes empresas

III. CARACTERÍSTICAS DE LOS GESTORES DE BASE DE DATOS NO RELACIONALES (NoSQL)

El crecimiento exponencial de los datos y la necesidad de manejarlos en formatos más flexibles impulsaron la aparición de los gestores de bases de datos NoSQL. Entre ellos, MongoDB destaca como uno de los más utilizados debido a su modelo documental, su escalabilidad horizontal y su capacidad de adaptarse a entornos donde los datos no siempre siguen una estructura rígida.

1. Definición de MongoDB

MongoDB es un sistema de gestión de bases de datos NoSQL, orientado a documentos, desarrollado en 2009 por MongoDB Inc. Su propósito es proporcionar un mecanismo de almacenamiento más flexible y escalable que los RDBMS tradicionales.

Características del modelo de MongoDB

- **Colecciones:** equivalen a las tablas en un sistema relacional, pero sin requerir esquemas predefinidos.
- **Documentos JSON/BSON:** cada documento puede representar objetos complejos y jerárquicos.
- **Consultas:** rapidez en consultas, tolerancia a fallos y facilidad de crecimiento.

2. Características Principales de MongoDB

Característica	Descripción
Modelo orientado a documentos	La información se organiza en documentos BSON con atributos anidados y arrays.
Esquema flexible	No es necesario definir un esquema rígido, lo que agiliza el desarrollo.
Escalabilidad horizontal	Implementa sharding para distribuir datos en múltiples servidores (Big Data).
Alto rendimiento	Optimizado para lecturas y escrituras rápidas.
Alta disponibilidad	Replica los datos en diferentes servidores mediante replica sets.
Compatibilidad multi-lenguaje	Java, Python, Node.js, C#, entre otros.
Herramientas de administración	Compass (interfaz gráfica), Atlas (servicio en la nube).

3. Evolución de MongoDB

Versión	Mejoras Incorporadas
Primeras versiones	Modelo documental y flexibilidad de esquemas.
MongoDB 3.x	Mayor estabilidad y seguridad con mejoras en la replicación.
MongoDB 4.x	Implementación de transacciones ACID para operaciones multidocumento.
MongoDB 5.x y 6.x	Integración de funciones de time-series, vistas materializadas y optimizaciones para la nube.
MongoDB Atlas	Servicio administrado en la nube pública con escalado y mantenimiento simplificados.

4. Ejemplo de Documento MongoDB (formato JSON)

```
// Colección: estudiantes
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nombre": "María Torres",
  "carrera": "Ingeniería de Sistemas",
  "notas": [
    { "curso": "Base de Datos", "nota": 18 },
    { "curso": "Algoritmos",   "nota": 20 }
  ],
  "activo": true,
  "fecha_ingreso": ISODate("2024-03-01")
}

// Consultar todos con nota > 17 en Base de Datos
db.estudiantes.find({
  "notas": { $elemMatch: { "curso": "Base de Datos", "nota": { $gt: 17 } } }
});
```

5. Comparación: MongoDB vs RDBMS

Aspecto	RDBMS (PostgreSQL/MySQL)	MongoDB (NoSQL)
Modelo de datos	Tablas con filas y columnas	Documentos JSON/BSON
Esquema	Rígido y predefinido	Flexible y dinámico
Escalabilidad	Vertical (más recursos al servidor)	Horizontal (más servidores)
Transacciones	ACID completo	ACID desde versión 4.x
Consultas	SQL estándar	API de MongoDB / MQL
Mejor uso	Datos estructurados, finanzas	Datos dinámicos, IoT, redes sociales

IV. COMPATIBILIDAD DE LOS DBMS CON DIFERENTES ARQUITECTURAS

La gestión de datos en el mundo actual exige que los Sistemas de Gestión de Bases de Datos (DBMS) sean compatibles con diversas arquitecturas que van desde entornos locales hasta infraestructuras en la nube y plataformas de Big Data.

1. Relacional vs. No Relacional según la Arquitectura

Relacionales (SQL Server, Oracle, MySQL, PostgreSQL)	No Relacionales (MongoDB, Cassandra, Redis, CouchDB)
• Se basan en el modelo de tablas con filas y columnas. • Ofrecen transacciones ACID (integridad y consistencia). • Adecuados para arquitecturas centralizadas y sistemas financieros. • Mejor compatibilidad con arquitecturas cliente-servidor.	• Se orientan a documentos, columnas, grafos o pares clave-valor. • Priorizan la escalabilidad horizontal y flexibilidad de esquemas. • Ideales para arquitecturas distribuidas y sistemas en tiempo real. • Mejor adaptabilidad a entornos de microservicios y la nube.

2. Factores de Elección: Escalabilidad, Rendimiento, Costo, Soporte y Comunidad

Factor	RDBMS	NoSQL
Escalabilidad	Escalabilidad vertical (más recursos al servidor)	Escalabilidad horizontal (más servidores en paralelo)
Rendimiento	Óptimo en operaciones transaccionales con datos estructurados	Supera en grandes volúmenes de datos semiestructurados
Costo	Oracle y SQL Server requieren licencias costosas	MySQL, PostgreSQL, MongoDB reducen costos iniciales
Soporte técnico	Soporte especializado garantizado en entornos críticos	Dependen en gran medida de la comunidad y terceros
Comunidad	Comunidades consolidadas con amplia documentación	PostgreSQL y MongoDB tienen comunidades muy activas

3. Tendencias Actuales en Compatibilidad con la Nube y Big Data

DBMS en la Nube

- **Amazon RDS, Google Cloud SQL, Azure SQL Database —** ofrecen RDBMS administrados con alta disponibilidad.
- **MongoDB Atlas —** facilita la implementación NoSQL con escalabilidad automática.
- **La nube proporciona —** elasticidad, facturación bajo demanda y gestión simplificada.

DBMS Híbridos

- **Oracle Multimodel, PostgreSQL con JSONB —** incorporan capacidades relacionales y no relacionales.
- **Trabajan con múltiples tipos de datos —** en una sola plataforma, reduciendo la fragmentación.

Compatibilidad con Big Data

- **Cassandra y HBase —** son compatibles con Hadoop y Spark para procesamiento masivo.
- **Data lakes y data warehouses —** en la nube amplían las posibilidades de análisis.
- **Microservicios y contenedores —** DBMS diseñados para Docker y Kubernetes aumentan flexibilidad.

V. FUNDAMENTOS DE SQL: GUÍA PRÁCTICA DE COMANDOS

SQL (Structured Query Language) es el lenguaje estándar para interactuar con bases de datos relacionales. Permite definir estructuras, manipular datos, controlar accesos y gestionar transacciones.

1. DDL – Data Definition Language (Definición de Datos)

Los comandos DDL permiten crear, modificar y eliminar la estructura de la base de datos.

```
-- ── CREATE: Crear tabla ──────────────────────────────────────
CREATE TABLE alumnos (
  id         INT          PRIMARY KEY AUTO_INCREMENT,
  nombre     VARCHAR(100) NOT NULL,
  email      VARCHAR(150) UNIQUE,
  carrera    VARCHAR(80),
  fecha_reg  DATE         DEFAULT CURRENT_DATE
);

-- ── ALTER: Modificar tabla (agregar columna) ─────────────────
ALTER TABLE alumnos ADD COLUMN telefono VARCHAR(20);

-- ── DROP: Eliminar tabla ─────────────────────────────────────
DROP TABLE IF EXISTS alumnos;

-- ── TRUNCATE: Vaciar tabla (sin eliminar estructura) ─────────
TRUNCATE TABLE alumnos;
```

2. DML – Data Manipulation Language (Manipulación de Datos)

Los comandos DML permiten insertar, consultar, actualizar y eliminar registros.

```
-- ── INSERT: Insertar registros ───────────────────────────────
INSERT INTO alumnos (nombre, email, carrera)
VALUES ('Carlos Pérez', 'carlos@uni.edu', 'Ingeniería');

-- ── SELECT: Consultar registros ──────────────────────────────
SELECT nombre, email
FROM   alumnos
WHERE  carrera = 'Ingeniería'
ORDER BY nombre ASC
LIMIT  10;

-- ── UPDATE: Actualizar registros ─────────────────────────────
UPDATE alumnos
SET    telefono = '987654321'
```

```
WHERE  id = 1;

-- ── DELETE: Eliminar registros ──────────────────────────────
DELETE FROM alumnos
WHERE  id = 5;
```

3. Consultas Avanzadas: JOIN, Subconsultas y Agregaciones

```
-- ── INNER JOIN: Combinar tablas ─────────────────────────────
SELECT a.nombre, c.nombre_curso, m.nota
FROM   matriculas m
INNER JOIN alumnos a  ON m.alumno_id = a.id
INNER JOIN cursos  c  ON m.curso_id  = c.id
WHERE  m.nota >= 11;

-- ── LEFT JOIN: Todos los alumnos, con o sin matrícula ──────────
SELECT a.nombre, COUNT(m.id) AS cursos_matriculados
FROM   alumnos a
LEFT JOIN matriculas m ON a.id = m.alumno_id
GROUP BY a.nombre
HAVING COUNT(m.id) > 0;

-- ── Subconsulta: Alumnos con nota mayor al promedio ────────────
SELECT nombre FROM alumnos
WHERE id IN (
  SELECT alumno_id FROM matriculas
  WHERE nota > (SELECT AVG(nota) FROM matriculas)
);

-- ── Funciones de agregación ────────────────────────────────────
SELECT carrera,
       COUNT(*)   AS total,
       AVG(nota)  AS promedio,
       MAX(nota)  AS mayor,
       MIN(nota)  AS menor
FROM   alumnos a
JOIN   matriculas m ON a.id = m.alumno_id
GROUP BY carrera
ORDER BY promedio DESC;
```

4. DCL y TCL – Control de Acceso y Transacciones

```
-- ── DCL: Gestión de permisos ────────────────────────────────
CREATE USER 'usuario1'@'localhost' IDENTIFIED BY 'password123';
GRANT SELECT, INSERT ON universidad.* TO 'usuario1'@'localhost';
REVOKE INSERT ON universidad.* FROM 'usuario1'@'localhost';
```

```
-- ── TCL: Control de transacciones ──────────────────────────────
BEGIN TRANSACTION;

UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 500 WHERE id = 2;

-- Si todo es correcto:
COMMIT;

-- Si ocurre un error, deshacer cambios:
ROLLBACK;
```

5. Índices, Vistas y Procedimientos Almacenados

```
-- ── ÍNDICE: Acelerar consultas ────────────────────────────────
CREATE INDEX idx_carrera ON alumnos(carrera);
CREATE UNIQUE INDEX idx_email ON alumnos(email);

-- ── VISTA: Consulta guardada como objeto ──────────────────────
CREATE VIEW vista_notas_altas AS
  SELECT a.nombre, c.nombre_curso, m.nota
  FROM   matriculas m
  JOIN   alumnos a ON m.alumno_id = a.id
  JOIN   cursos  c ON m.curso_id  = c.id
  WHERE  m.nota >= 15;

SELECT * FROM vista_notas_altas;

-- ── PROCEDIMIENTO ALMACENADO (MySQL) ──────────────────────────
DELIMITER //
CREATE PROCEDURE registrar_alumno(
  IN p_nombre VARCHAR(100),
  IN p_email  VARCHAR(150),
  IN p_carrera VARCHAR(80)
)
BEGIN
  INSERT INTO alumnos (nombre, email, carrera)
  VALUES (p_nombre, p_email, p_carrera);
  SELECT 'Alumno registrado exitosamente' AS mensaje;
END //
DELIMITER ;

-- Llamar al procedimiento:
CALL registrar_alumno('Ana López', 'ana@uni.edu', 'Sistemas');
```

VI. EJERCICIOS PROPUESTOS

Ejercicio 1 – Cuadro Comparativo de DBMS

Diseña un cuadro comparativo entre un DBMS relacional (PostgreSQL), un NoSQL (MongoDB) y un híbrido (ArangoDB). Incluye: modelo de datos, lenguaje de consulta, ventajas, limitaciones y escenarios de uso.

Criterio	PostgreSQL	MongoDB	ArangoDB
Modelo de datos	Tablas relacionales	Documentos JSON	Multi-modelo (doc + grafo)
Lenguaje de consulta	SQL estándar	MQL / Aggregation	AQL (ArangoDB Query)
Escalabilidad	Vertical + horizontal limitado	Horizontal (sharding)	Horizontal y vertical
Transacciones	ACID completo	ACID desde v4.x	ACID completo
Ventajas	Estándares, extensibilidad	Flexibilidad, velocidad	Versatilidad multi-modelo
Limitaciones	Esquema rígido	Joins complejos	Menor comunidad
Escenario de uso	Finanzas, análisis, ciencia	IoT, redes sociales, e-commerce	Grafos + documentos

Ejercicio 2 – Propuesta de Proyecto: Sistema de Ventas en Línea

Escenario

Un startup necesita crear un sistema para gestionar ventas en línea y almacenar datos de clientes, productos e interacciones en redes sociales.

Elabora una propuesta indicando:

- Qué tipo de DBMS elegirías (relacional, no relacional o híbrido).
- Justificación con al menos tres argumentos (rendimiento, escalabilidad, estructura de datos).

Ejercicio 3 – Diseño de Base de Datos Académica con SQL

Diseña una base de datos para un sistema académico que gestione estudiantes, cursos y notas. Implementa las siguientes tablas y consultas:

```sql
-- Tablas base del sistema académico
CREATE TABLE facultades (
  id     INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL
);
```

```sql
CREATE TABLE alumnos (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  nombre      VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE,
  facultad_id INT,
  FOREIGN KEY (facultad_id) REFERENCES facultades(id)
);

CREATE TABLE cursos (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  nombre_curso VARCHAR(100) NOT NULL,
  creditos    INT DEFAULT 3
);

CREATE TABLE matriculas (
  id         INT PRIMARY KEY AUTO_INCREMENT,
  alumno_id  INT,
  curso_id   INT,
  nota       DECIMAL(4,2),
  semestre   VARCHAR(10),
  FOREIGN KEY (alumno_id) REFERENCES alumnos(id),
  FOREIGN KEY (curso_id)  REFERENCES cursos(id)
);

-- Consulta: Reporte de calificaciones por facultad
SELECT f.nombre AS facultad, a.nombre AS alumno,
       c.nombre_curso, m.nota, m.semestre
FROM   matriculas m
JOIN   alumnos a   ON m.alumno_id  = a.id
JOIN   cursos  c   ON m.curso_id   = c.id
JOIN   facultades f ON a.facultad_id = f.id
ORDER BY f.nombre, a.nombre;
```

Ejercicio 4 – MongoDB: Modelo Documental para Biblioteca

Diseña la base de datos de una biblioteca universitaria usando MongoDB, en la que se gestionen libros, autores, préstamos y usuarios.

```mongodb
// Colección: libros
{
  "_id": ObjectId("..."),
  "titulo": "Fundamentos de Bases de Datos",
  "autores": ["Silberschatz", "Korth", "Sudarshan"],
  "isbn": "978-0-07-352332-3",
  "anio": 2019,
  "categorias": ["Bases de Datos", "Ciencias de la Computación"],
  "disponible": true
}

// Colección: prestamos
{
  "_id": ObjectId("..."),
```

```
  "usuario_id": ObjectId("..."),
  "libro_id": ObjectId("..."),
  "fecha_prestamo": ISODate("2025-04-01"),
  "fecha_devolucion": ISODate("2025-04-15"),
  "devuelto": false
}

// Consultar libros disponibles de una categoría
db.libros.find({
  "categorias": "Bases de Datos",
  "disponible": true
}).sort({ "titulo": 1 });
```

VII. CONCLUSIONES

El estudio de los gestores de bases de datos evidencia que son más que herramientas técnicas: representan la infraestructura invisible que sostiene el funcionamiento de la economía digital y de los servicios esenciales de la sociedad.

Conclusiones Clave

1. **Los RDBMS siguen siendo fundamentales** en entornos estructurados y críticos como finanzas, gobierno y educación.
2. **Los NoSQL destacan** en arquitecturas modernas, distribuidas y orientadas a Big Data e IoT.
3. **La tendencia actual** combina ambos enfoques en entornos híbridos y en la nube.
4. **La elección del DBMS** dependerá de factores como costos, volumen de datos, integración tecnológica y objetivos del proyecto.
5. **SQL sigue siendo** el lenguaje universal más importante para la gestión y consulta de datos estructurados.

Glosario de Términos Clave

Término	Definición
DBMS	Database Management System – Sistema de gestión de bases de datos.
RDBMS	Relational DBMS – Gestores que organizan datos en tablas relacionadas.
NoSQL	Not Only SQL – Gestores para datos no estructurados o semiestructurados.
SQL	Structured Query Language – Lenguaje estándar para bases de datos relacionales.
ACID	Atomicidad, Consistencia, Aislamiento, Durabilidad – propiedades de transacciones.
Sharding	Partición horizontal de datos en múltiples servidores para escalabilidad.
Replica Set	Conjunto de servidores MongoDB que mantienen copias de los datos.
DDL	Data Definition Language – Comandos para definir estructuras (CREATE, ALTER, DROP).
DML	Data Manipulation Language – Comandos para manipular datos (INSERT, SELECT, UPDATE, DELETE).
DCL	Data Control Language – Comandos para gestionar permisos (GRANT, REVOKE).
TCL	Transaction Control Language – Comandos de transacciones (COMMIT, ROLLBACK).
Índice	Estructura que acelera las búsquedas en una columna o conjunto de columnas.
Vista (VIEW)	Consulta guardada que se comporta como una tabla virtual.
JOIN	Operación para combinar filas de dos o más tablas basándose en columnas relacionadas.